

COMMON ASSIGNMENT REPORT

AI-POWERED FAKE NEWS AND MISINFORMATION

DETECTION SYSTEM USING RANDOM FOREST

ALGORITHM

Submitted as part of the Common Assignment

G AMRUTHA

192511250

CSA1706-ARTIFFICIAL INTELLIGENCE

COMPUTER SCIENCE ENGINEERING

SIMATS ENGINEERING

Academic Year: 2026–2027

1. Problem Statement and Problem Formulation

The rapid growth of social media, online news platforms, blogs, and digital communication has made it easy for false and misleading information to spread quickly. Fake news can influence public opinion, create confusion, and reduce trust in reliable information sources.

Manual verification of every news article is time-consuming and difficult because of the large volume of information generated every day. Therefore, an automated system is required to analyze news content and predict whether it is likely to be real or fake.

This project proposes an AI-powered Fake News and Misinformation Detection System using the Random Forest algorithm. The system processes the textual content of a news article, extracts useful features using Natural Language Processing (NLP) and TF-IDF, and uses a Random Forest classifier to classify the news.

Problem formulation:

Input: news title/content and optional metadata.

Processing: preprocessing → TF-IDF feature extraction → Random Forest classification.

Output: REAL NEWS or FAKE NEWS.

2. Objective and Expected Outcomes

The primary objective is to develop an automated machine-learning system capable of classifying news articles as real or fake. The project demonstrates the application of NLP, feature extraction, supervised learning, ensemble classification, and performance evaluation.

- Develop an AI-based system for detecting fake news.
- Preprocess and analyze news articles using NLP techniques.
- Extract useful textual features using TF-IDF.
- Implement the Random Forest classification algorithm.
- Classify news articles as real or fake.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Provide a simple interface for prediction.

3. Requirements, Constraints and Assumptions

3.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	Minimum 5 GB free space
System	Laptop/Desktop
Internet	Required for dataset/library installation

3.2 Software Requirements

Software/Library	Purpose
Python	Programming language
Jupyter Notebook / VS Code	Development
Pandas	Data processing
NumPy	Numerical operations
Scikit-learn	Machine learning
NLTK	NLP preprocessing
Matplotlib	Visualization
Seaborn	Confusion matrix visualization
Flask/Streamlit	Web interface
GitHub	Source-code submission

3.3 Constraints

- Prediction depends on the quality and representativeness of the training dataset.
- The model may not correctly identify new misinformation patterns not represented in training data.
- Text classification alone cannot guarantee that a claim is factually true.
- Biased or incomplete training data can affect predictions.
- Very short articles may not provide enough information for reliable classification.
- The system should be treated as a decision-support tool rather than a replacement for professional fact-checking.

3.4 Assumptions

- The dataset contains labelled real and fake news examples.
- The news text is available in a supported language.
- The dataset is sufficiently large for model training.
- Both classes are reasonably represented.
- Extracted features contain useful patterns for classification.

4. Application of Relevant Course Knowledge / Concepts

- Artificial Intelligence: automated classification and decision-making.
- Machine Learning: supervised learning, classification, training, testing, and evaluation.
- Data Structures and Algorithms: decision trees, ensemble learning, and feature representation.
- Natural Language Processing: tokenization, stop-word removal, normalization, and TF-IDF.
- Software Engineering: modular design, implementation, testing, and documentation.

5. Design / Proposed Solution / Methodology

The proposed system contains a data preparation stage, NLP preprocessing stage, TF-IDF feature extraction stage, Random Forest training stage, prediction stage, and evaluation stage.

System flow: User News Input → Text Preprocessing → TF-IDF Vectorization → Random Forest Classifier → Prediction → Real/Fake.

5.1 Data Collection

A labelled fake-news dataset is required. A typical dataset contains news text and a corresponding label such as REAL or FAKE. The dataset is divided into training and testing sets, for example 80% training and 20% testing.

5.2 Text Preprocessing

1. Convert text to lowercase.
2. Remove punctuation and special characters.
3. Remove unnecessary numbers and extra spaces.
4. Remove stop words.
5. Tokenize the text.

5.3 TF-IDF Feature Extraction

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors. It gives higher importance to terms that are useful for distinguishing documents.

$$\text{TFIDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

5.4 Random Forest Classifier

Random Forest is an ensemble supervised-learning algorithm that combines predictions from multiple decision trees. Each tree produces a class prediction, and the final class is selected through aggregation/majority voting. This approach is suitable for classification and can reduce dependence on the decisions of a single tree.

6. Algorithm / Pseudocode / Flowchart

6.1 Random Forest Algorithm

6. Load the labelled news dataset.
7. Separate news text/features and target labels.
8. Clean and preprocess the news text.
9. Convert the text into TF-IDF vectors.
10. Split the data into training and testing sets.
11. Create multiple decision trees using the Random Forest classifier.
12. Train the Random Forest using the training vectors.
13. Generate predictions from the individual trees.
14. Combine predictions using majority voting.
15. Evaluate the model using suitable performance metrics.
16. For a new article, apply the same preprocessing and TF-IDF transformation and obtain the predicted class.

6.2 Pseudocode

Algorithm: Fake News Detection Using Random Forest

Input: Labelled news dataset D

1. Load D.
2. Remove missing/duplicate records where appropriate.
3. Set X = news text and Y = labels.
4. Preprocess X: lowercase, clean, tokenize, and remove stop words.
5. Transform X into TF-IDF vectors.
6. Split the dataset into training and testing sets.
7. Initialize Random Forest classifier.
8. Train the classifier on the training data.
9. Predict labels for the test data.
10. Calculate accuracy, precision, recall, and F1-score.
11. For a new news article, preprocess and vectorize it.
12. Predict REAL or FAKE.

Output: Predicted news class.

6.3 Flowchart

START



Load Dataset



Clean Dataset



Text Preprocessing



TF-IDF Feature Extraction



Train/Test Split



Random Forest Training



Model Testing



Performance Evaluation



Enter New News



Prediction



REAL / FAKE

↓
END

7. Implementation / Source Code and Environment / Tools Used

The implementation can be developed in Python using Pandas, NumPy, Scikit-learn, NLTK, Matplotlib, and Seaborn. A Flask or Streamlit interface can be added for user interaction.

7.1 Core Python Implementation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
df = pd.read_csv("fake_news_dataset.csv")

# Features and target
X = df["text"]
y = df["label"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# TF-IDF
vectorizer = TfidfVectorizer(
    stop_words="english",
    max_features=5000
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Random Forest
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)

model.fit(X_train_tfidf, y_train)

# Prediction
y_pred = model.predict(X_test_tfidf)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

8. Test Cases and Expected/Actual Results

Test Case	Input	Expected Result	Actual Result
TC01	Known genuine news article	REAL	To be recorded after execution
TC02	Known fabricated news article	FAKE	To be recorded after execution
TC03	Very short article	Prediction may be uncertain	To be recorded after execution
TC04	Misleading/sensational article	FAKE or dataset-defined class	To be recorded after execution
TC05	Previously unseen article	REAL or FAKE prediction	To be recorded after execution

9. Results and Validation

The final report should contain the actual experimental results obtained from the selected dataset. Do not invent accuracy or confidence values. Record the values generated by the executed implementation.

Metric	Value
Accuracy	Fill with actual result
Precision	Fill with actual result
Recall	Fill with actual result
F1-score	Fill with actual result

Recommended evidence includes a confusion matrix, class-distribution graph, metric comparison graph, and sample prediction outputs.

10. Analysis, Comparison, Trade-offs and Justification

Random Forest is selected because it combines multiple decision trees instead of relying on a single tree. It is a practical ensemble approach for classification and can work with many extracted features. Its main trade-offs are computational cost and dependence on feature quality and training data.

Algorithm	Main Advantage	Main Limitation
Naive Bayes	Fast and simple	Simplifying assumptions
Logistic Regression	Simple and interpretable	Linear decision boundary
Decision Tree	Easy to visualize	Can overfit
Random Forest	Robust ensemble classification	Higher computation than a single tree
SVM	Strong performance on many text tasks	Can be computationally expensive at scale
LSTM	Can model sequential patterns	Higher training complexity
BERT	Strong contextual language representation	Higher computational requirements

Random Forest is appropriate for this project because it provides a clear machine-learning workflow while allowing performance analysis, tree-based reasoning, and feature-importance analysis. Nevertheless, classification should not be interpreted as definitive proof that an article is factually true.

11. Broader Considerations / SDG Relevance

11.1 Responsible AI Considerations

- Avoid biased training datasets.
- Protect user-submitted content and personal information.
- Explain that predictions are model outputs, not absolute facts.
- Report uncertainty and model limitations.
- Encourage verification of important claims through reliable sources.

11.2 SDG Relevance

- SDG 4 – Quality Education: supports digital and information literacy.
- SDG 9 – Industry, Innovation and Infrastructure: applies AI and machine learning to an information-quality problem.
- SDG 16 – Peace, Justice and Strong Institutions: supports access to more reliable information and trustworthy digital communication.

12. Conclusion, Limitations and Possible Improvements

The AI-Powered Fake News and Misinformation Detection System using Random Forest provides an automated approach for classifying news articles based on patterns learned from labelled data. The proposed system combines Natural Language Processing, TF-IDF feature extraction, and Random Forest classification. It demonstrates how supervised machine learning can be applied to a real-world information-quality problem.

12.1 Limitations

- Performance depends on dataset quality and representativeness.
- New misinformation patterns may not be recognized.
- Sarcasm and satire can be difficult to classify.
- Multilingual content may require language-specific preprocessing.
- The model does not independently establish factual truth.
- Dataset bias can affect predictions.

12.2 Possible Improvements

17. Integrate BERT or other transformer-based models.
18. Support multilingual fake-news detection.
19. Add source-credibility analysis.

20. Add claim extraction and evidence retrieval.
21. Develop real-time news analysis.
22. Add explainable AI to show important factors behind predictions.
23. Maintain a database of previously verified claims.
24. Explore hybrid Random Forest and deep-learning architectures.

13. Individual Contribution of Group Members

Member	Suggested Contribution
Member 1	Dataset collection, cleaning, NLP preprocessing, TF-IDF
Member 2	Random Forest implementation, training, tuning, evaluation
Member 3	Interface, testing, screenshots, report, GitHub documentation

Modify this table to reflect the actual work performed by each team member.

14. One-Page Individual Reflection

This project provided an opportunity to understand how Artificial Intelligence and Machine Learning can be applied to a real-world problem such as fake news and misinformation detection. During the development of the system, I learned how raw textual information can be transformed into machine-readable features using Natural Language Processing and TF-IDF.

One of the major design decisions was selecting the Random Forest algorithm. Random Forest was selected because it combines multiple decision trees and provides a practical approach for classification. I also learned the importance of dividing the dataset into training and testing data and evaluating the model using accuracy, precision, recall, and F1-score.

A major challenge was understanding how unstructured news text can be converted into useful numerical features. Another important learning was that a machine-learning model does not automatically know whether a statement is factually true; its prediction depends on patterns learned from the training data.

Through this project, I gained practical knowledge of data preprocessing, feature extraction, supervised learning, model training, testing, and evaluation. I also learned that responsible AI development requires consideration of bias, privacy, transparency, and model limitations.

The project has relevance to SDG 4, SDG 9, and SDG 16 through digital literacy, technological innovation, and reliable information. Overall, this assignment improved my understanding of how theoretical machine-learning concepts can be converted into a practical AI application.

15. References

25. Scikit-learn documentation – Random Forest Classifier.
26. Scikit-learn documentation – TfidfVectorizer.
27. Python documentation.
28. NLTK documentation.
29. The exact fake-news dataset used for the implementation.
30. Research papers related to fake-news detection and machine learning.
31. Documentation for any external libraries, tools, datasets, or code used.

Note: Replace the generic reference descriptions with the exact URLs/DOIs and dataset details used in the final implementation.

16. Submission Checklist

- Complete report
- Python source code
- Dataset and dataset citation
- Random Forest implementation
- Test cases
- Execution screenshots
- Actual accuracy/precision/recall/F1 results
- Confusion matrix and graphs
- Individual contribution
- One-page individual reflection
- References
- GitHub repository link